

Алгоритм построения ДКА из регулярного выражения

Гафаров Т.Э. Б01-411

Что такое регулярные выражения, НКА и ДКА?

- *Регулярные выражения* - это способ описания паттернов в тексте. Они помогают найти определенные последовательности символов в строках. Например, с помощью регулярного выражения можно найти все вхождения слов в тексте или проверить, состоит ли строка только из цифр.
- *Недетерминированный конечный автомат (НКА)* - это модель, которая помогает проверять, подходит ли строка под регулярное выражение. В НКА из одного состояния можно переходить в несколько разных состояний по одному и тому же символу.
- *Детерминированный конечный автомат (ДКА)* - это улучшенная версия НКА, в которой для каждого состояния и символа есть только один переход. То есть, из каждого состояния по каждому символу возможен только один путь.
- ДКА проще и быстрее, чем НКА, потому что не нужно выбирать между несколькими возможными путями.

Постановка задачи

От регулярного выражения к детерминированному автомату

- Проблема:
Проверка строки по регулярному выражению может быть медленной.
НКА допускает неоднозначность и ϵ -переходы $\rightarrow$ сложно реализовать эффективно.
- Цель:
Построить ДКА — автомат, где из каждого состояния по каждому символу есть не более одного перехода.
- Зачем:
Проверка строки выполняется за линейное время $O(n)$.
- Этапы решения:
 - 1) Регулярное выражение $\rightarrow$ НКА
(алгоритм Томпсона)
 - 2) НКА $\rightarrow$ ДКА
 - 3) ДКА $\rightarrow$ минимальный ДКА

Построение НКА по регулярному выражению

Дано: регулярное выражение

Задача: построить недетерминированный автомат, распознающий те же строки.

Метод: алгоритм Томпсона

- Для каждого символа создаётся атомарный автомат (2 состояния)
- Операции $|$, $;$, $*$ объединяют автоматы ϵ -переходами
- Размер НКА пропорционален длине выражения ($\leq 2n$)

Результат: НКА с одним стартовым и одним финальным состоянием, потенциально с несколькими переходами по одному символу.

Алгоритм Томпсона для построения НКА

Строится рекурсивно по структуре выражения:

1) Базовые автоматы

- Для символа $s \in \Sigma$:
строится автомат из двух состояний $q_0 \rightarrow q_1$ с переходом по s .
 q_0 – начальное, q_1 – конечное.
- Для пустого символа ϵ (используется редко):
автомат из одного состояния, которое одновременно начальное и конечное.

2) Составные операции

Пусть у нас уже есть НКА A для R_1 и НКА B для R_2 .

• Конкатенация ($R_1 \cdot R_2$):

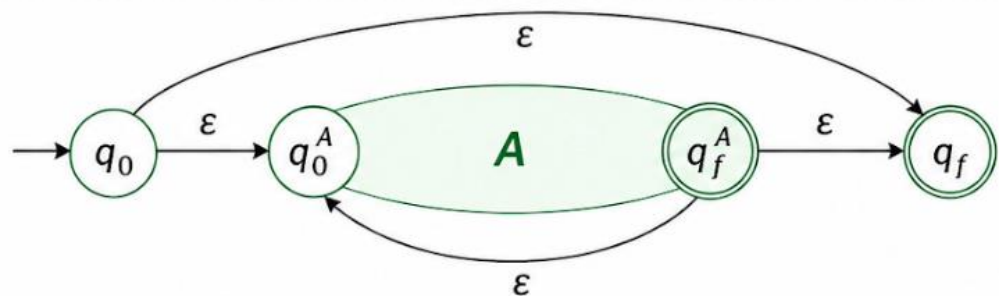
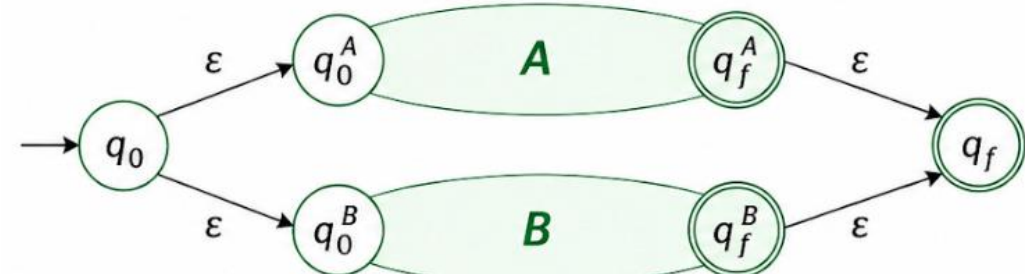
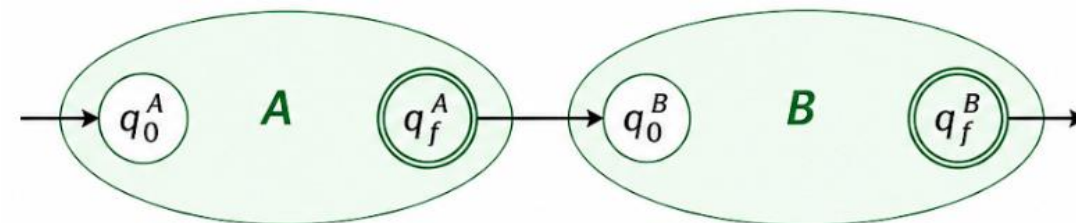
- Добавляем ϵ -переход из каждого финального состояния A в начальное состояние B .
- Начало результирующего автомата = начало A .
- Финалы = финалы B .

• Альтернатива ($R_1 \mid R_2$):

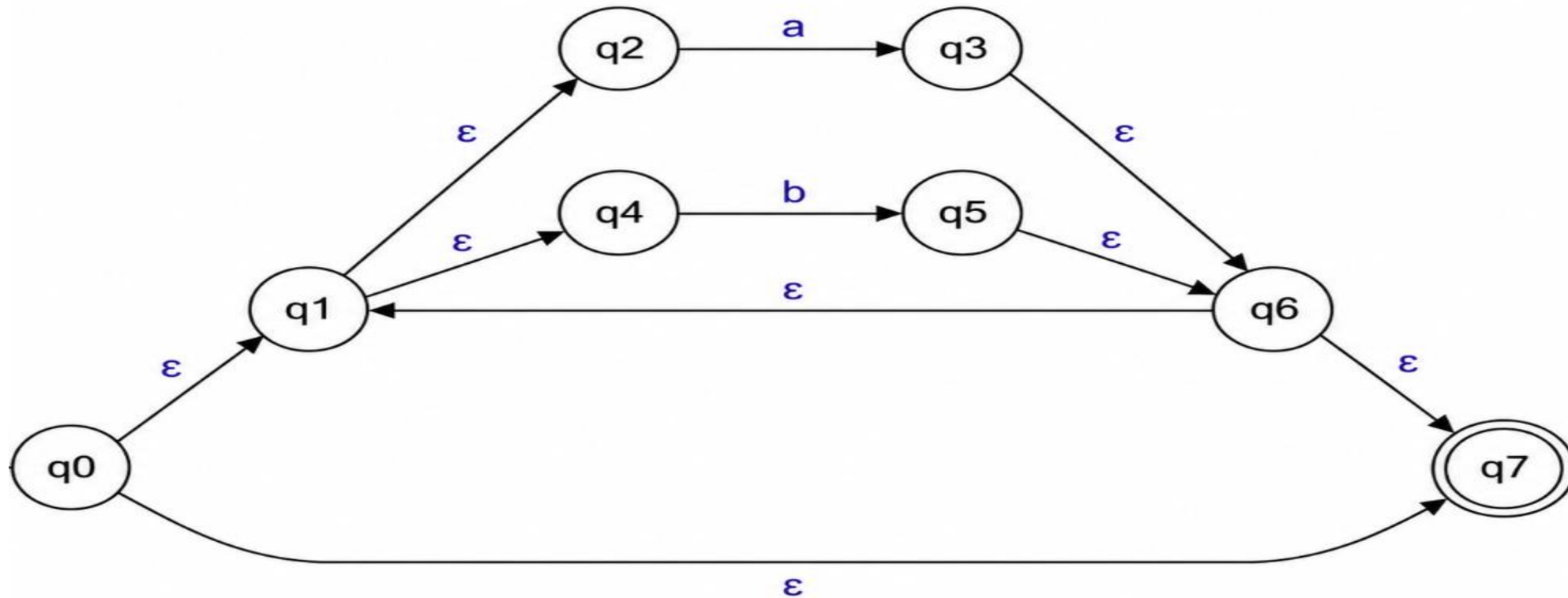
- Создаём новое начальное состояние q_{start} .
- Проводим ϵ -переходы $q_{\text{start}} \rightarrow$ начало A и $q_{\text{start}} \rightarrow$ начало B .
- Создаём новое финальное состояние q_{final} .
- Из каждого финального состояния A и B проводим ϵ -переход в q_{final} .

• Звезда Клини (R^*):

- Создаём новое начальное состояние q_{start} и финальное q_{final} .
- ϵ -переход $q_{\text{start}} \rightarrow$ начало A (вход в тело).
- ϵ -переход $q_{\text{start}} \rightarrow q_{\text{final}}$ (возможность пустой строки).
- Из каждого финала A ϵ -переход обратно в начало A (повторение).
- Из каждого финала A ϵ -переход в q_{final} (выход).



Пример: НКА для $(a|b)^*$, построенный по Томпсону.



НКА $\rightarrow$ ДКА (метод подмножеств)

Основная идея:

Каждое состояние ДКА соответствует множеству состояний НКА, в которых автомат может находиться одновременно.

Операция

$$C_{\varepsilon}(R)$$

$$move(R, a)$$

Смысл

ε -замыкание — все состояния, достижимые из R по ε -переходам (включая сами состояния R)

Переход по символу a из любого состояния множества R

Алгоритм построения ДКА по НКА

Вход: НКА $(\Sigma, Q, Q_f, \delta, q_0)$

Выход: ДКА $(\Sigma, Q', Q'_f, D, q'_0)$

Шаг 1. Начальное состояние

$q'_0 = \text{C} \in (\{q_0\})$

Шаг 2. Основной цикл

Пока существует непомеченное состояние $R \in Q'$:

1. Пометить состояние R

2. Для каждого символа $a \in \Sigma$ выполнить:

$S = \text{C} \in (\text{move}(R, a))$

3. Если $S \neq \emptyset$:

- Если $S \notin Q'$:

добавить S в Q' как непомеченное

- Добавить переход в ДКА:

$D(R, a) = S$

Шаг 3. Финальные состояния

$Q'_f = \{ S \in Q' \mid S \cap Q_f \neq \emptyset \}$

Пример: преобразование НКА в ДКА

1. Инициализация

Стартовое множество: ϵ -замыкание($\{1\}$) = $\{1\}$ (ϵ -переходов нет)

Очередь $Q = [\{1\}]$

Состояния ДКА: пока нет

2. Обрабатываем $\{1\}$ (достаём из очереди, Q становится пустой)

• Символ a :

$$\delta(\{1\}, a) = \delta(1, a) = \{1, 2\}$$

$$\epsilon\text{-замыкание}(\{1, 2\}) = \{1, 2\}$$

Это новое множество $\rightarrow$ добавляем $\{1, 2\}$ в очередь и в ДКА.

Переход ДКА: $\{1\} \xrightarrow{a} \{1, 2\}$

• Символ b :

$$\delta(\{1\}, b) = \{1\}$$

$$\epsilon\text{-замыкание}(\{1\}) = \{1\}$$

Такое множество уже есть (это $\{1\}$), ничего не добавляем.

Переход ДКА: $\{1\} \xrightarrow{b} \{1\}$

Текущая очередь: $[\{1, 2\}]$

3. Обрабатываем $\{1, 2\}$ (достаём, очередь пуста)

• Символ a :

$$\delta(\{1, 2\}, a) = \delta(1, a) \cup \delta(2, a) = \{1, 2\} \cup \{2\} = \{1, 2\}$$

$$\epsilon\text{-замыкание}(\{1, 2\}) = \{1, 2\}$$

Множество уже известно, переход: $\{1, 2\} \xrightarrow{a} \{1, 2\}$

• Символ b :

$$\delta(\{1, 2\}, b) = \delta(1, b) \cup \delta(2, b) = \{1\} \cup \{2\} = \{1, 2\}$$

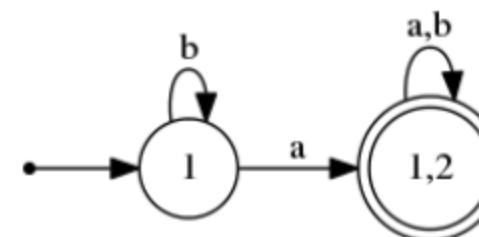
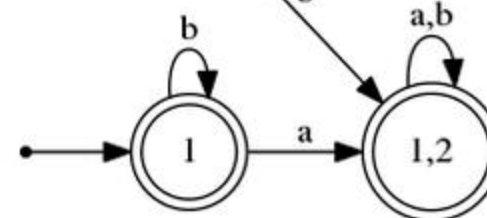
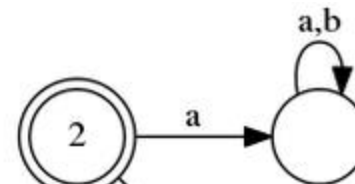
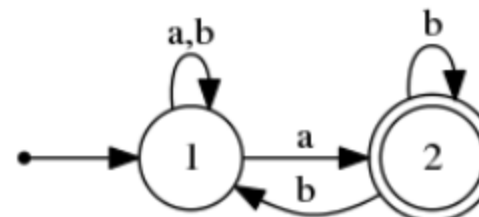
Переход: $\{1, 2\} \xrightarrow{b} \{1, 2\}$

Очередь пуста – построение закончено.

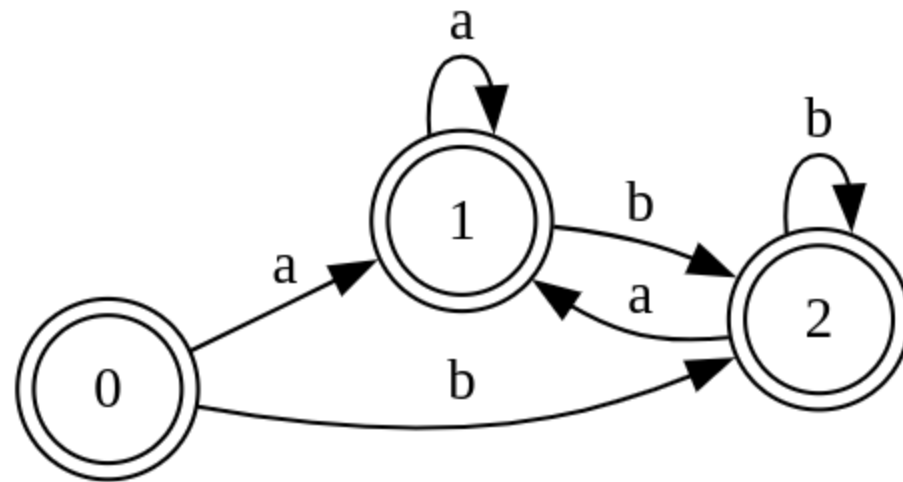
4. Финальные состояния ДКА

Финальным считается любое множество, содержащее конечное состояние НКА (2).

У нас такое множество: $\{1, 2\} \rightarrow$ оно и будет финальным.



Пример: НКА в ДКА для $(a|b)^*$



Минимизация ДКА

- Цель: Уменьшить количество состояний ДКА, полностью сохранив принимаемый язык.
 - Зачем это нужно?
 - Ускоряет обработку строк (меньше переходов)
 - Сокращает объём памяти, занимаемой автоматом
 - Методы:
 1. Метод таблицы эквивалентности
Итеративно помечает различимые пары состояний.
Время: $O(|\Sigma| \cdot n^2)$, Память: $O(n^2)$
 2. Алгоритм Хопкрофта
Разбивает состояния на классы эквивалентности за счёт обратных переходов.
Время: $O(|\Sigma| \cdot n \log n)$, Память: $O(|\Sigma| \cdot n)$

Минимизация ДКА

Цель: Объединить одинаковые состояния, не изменяя язык автомата.

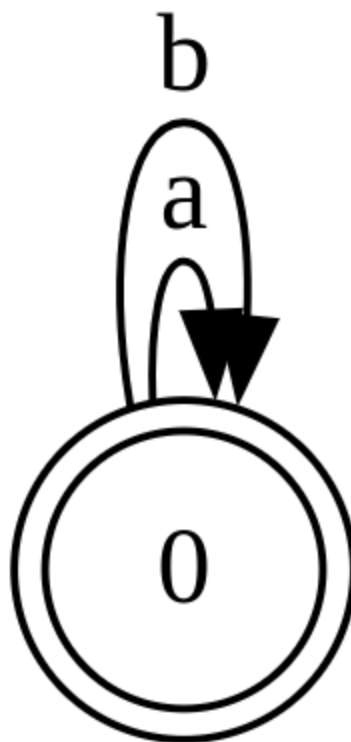
Состояния p и q различимы, если:

- одно из них финальное, а другое нет;
- или по некоторому символу они переходят в уже различимые состояния.

Алгоритм минимизации:

- Построить таблицу всех пар состояний.
- Сразу отметить пары:
 - финальное или нефинальное.
- Проверять остальные пары:
 - если по некоторому символу переходы ведут в различимые состояния, то текущая пара тоже различима.
- Повторять процесс, пока новые отметки не перестанут появляться.
- Все непомеченные пары объединить в одно состояние.

Пример: минимизация для $(a|b)^*$



Оценка сложности

Этап	Время (худший случай)	Память
Регулярное в НКА	$O(k \cdot n)$, где n - длина выражения, k - размер алфавита	$O(n)$
НКА в ДКА	$O(2^s \cdot k)$, где s - число состояний НКА	$O(2^s)$ (размер результирующего ДКА)
Минимизация ДКА	$O(k \cdot m^2)$ $O(k \cdot m \log m)$	$O(m^2)$ $O(k \cdot m)$

Бенчмаркинг

Длина	NFA	DFA	Минимизация	Общее
10	0.01 мс	0.03 мс	0.02 мс	0.07 мс
100	0.07 мс	0.93 мс	0.29 мс	1.29 мс
1000	0.25 мс	71.9 мс	4.15 мс	76.3 мс
10000	18.1 мс	58.9 с	0.80 с	59.7 с

Полезные ссылки

- [Преобразование регулярного выражения в ДКА](#)
- [Алгоритм преобразования НКА в эквивалентный ДКА](#)
- [Конечные автоматы и регулярные выражения](#)
- [алгоритм Томпсона](#)
- [Минимизация ДКА, алгоритм за \$O\(n^2\)\$](#)
- [Минимизация ДКА, алгоритм Хопкрофта \(сложность \$O\(n \log n\)\$ \)](#)
- [Репозиторий с бенчмарками](#)